Project - High Level Design

on

Chat with Technology Documents

Course Name: DevOps

Institution Name: Medicaps University – Datagami Skill Based Course

Student Name(s) & Enrolment Number(s):

Sr no	Student Name	Enrolment Number
01	Aayush Pal	EN22CS301022
02	Aayushee Dhote	EN22CS301024
03	Aditi Shrestha	EN22CS301044
04	Aditya Chouhan	EN22CS301049
05	Aditya Singh Chouhan	EN22CS301066

Group Name: G9D1

Project Number: GAI-09

Industry Mentor Name:

University Mentor Name: Dr. Ritesh Joshi

Academic Year: 2025-2026

Table of Contents

1. Introduction.
   - 1.1. Scope of the document.
   - 1.2. Intended Audience
   - 1.3. System overview.
2. System Design.
   - 2.1. Application Design
   - 2.2. Process Flow.
   - 2.3. Information Flow.
   - 2.4. Components Design
   - 2.5. Key Design Considerations
   - 2.6. API Catalogue.
3. Data Design
   - 3.1. Data Model
   - 3.2. Data Access Mechanism
   - 3.3. Data Retention Policies
   - 3.4. Data Migration
4. Interfaces
5. State and Session Management
6. Caching
7. Non-Functional Requirements
   - 7.1 Security Aspects
   - 7.2 Performance Aspects
8. References

1. Introduction

This document presents the High-Level Design (HLD) of a web-based application developed using the Flask framework and deployed using containerization and orchestration technologies. The primary objective of this system is to demonstrate modern deployment practices by integrating container-based packaging and automated orchestration.
The application is containerized using Docker, which ensures portability and consistency across different environments. The deployment is managed using Kubernetes, which provides scalability, self-healing, and efficient resource utilization. The entire system is hosted on a cloud-based virtual machine provided by Amazon Web Services (EC2 instance).
This project highlights how a locally developed application can be transformed into a production-like deployment using industry-standard tools and practices.

1.1 Scope of the Document

This document outlines the overall architecture, components, and deployment strategy of the system. It provides a high-level understanding of how the application is built, containerized, deployed, and accessed by end users.

The scope includes:

- Description of system architecture and components
- Overview of containerization using Docker
- Deployment strategy using Kubernetes
- Interaction between application, container, and orchestration layers
- High-level process flow of the system

This document does not cover low-level implementation details such as source code logic, database schema, or detailed API specifications.

1.2 Intended Audience

This document is intended for:

- Developers who want to understand the deployment architecture of the application
- Students and learners studying cloud computing and DevOps practices
- System administrators responsible for managing deployments

- Evaluators or instructors reviewing the project implementation

The document assumes basic knowledge of web applications, containerization, and cloud platforms.

1.3 System Overview

The system is a web-based application developed using the Flask framework, designed to run inside a containerized environment. The application is packaged into a Docker image, which includes all required dependencies and runtime configurations.
This containerized application is deployed on an EC2 instance and managed using Kubernetes. Kubernetes handles the deployment through a Deployment configuration that maintains multiple instances (replicas) of the application for high availability. A Service of type NodePort is used to expose the application to external users.
Users access the application through a web browser using the public IP address of the EC2 instance along with the assigned port number. Kubernetes ensures that the application remains available by automatically restarting failed containers and maintaining the desired number of running instances.
Overall, the system demonstrates a complete deployment pipeline involving application development, containerization, cloud hosting, and orchestration.

2. System Design:

The system design describes the overall structure and functioning of the application, including its components, workflows, and interactions. The system is designed using a modular approach where each layer performs a specific role, ensuring scalability, maintainability, and efficient deployment.
The application follows a container-based architecture, where the Flask web application is packaged using Docker and deployed using Kubernetes on an Amazon Web Services EC instance.

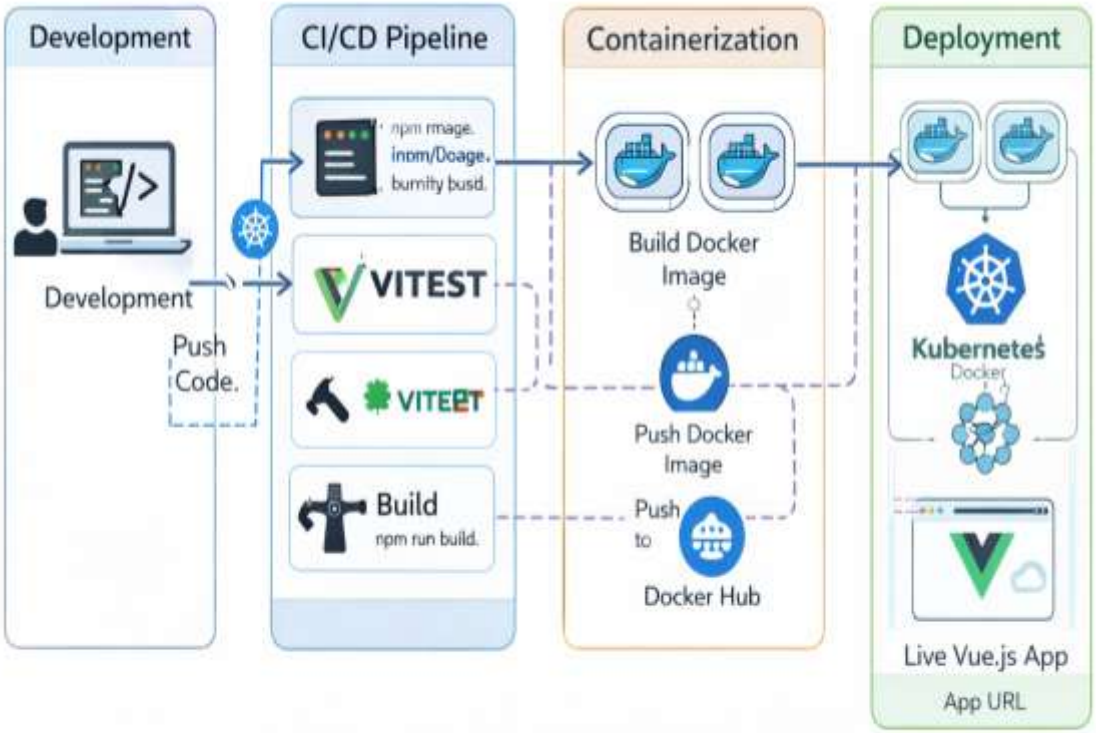
2.1Application Design

The application is developed using the Flask framework, following a simple and modular structure. It consists of:

- A main application file (app.py) that handles routing and request processing
- Templates for rendering user interfaces
- Static files such as CSS and JavaScript for frontend styling and behavior

The application runs as a web server that listens for incoming HTTP requests and returns appropriate responses. It is designed to be lightweight and easily deployable inside a container.
The application is configured to run on 0.0.0.0, allowing external access when deployed in containerized environments.

2.2 Architecture Diagram



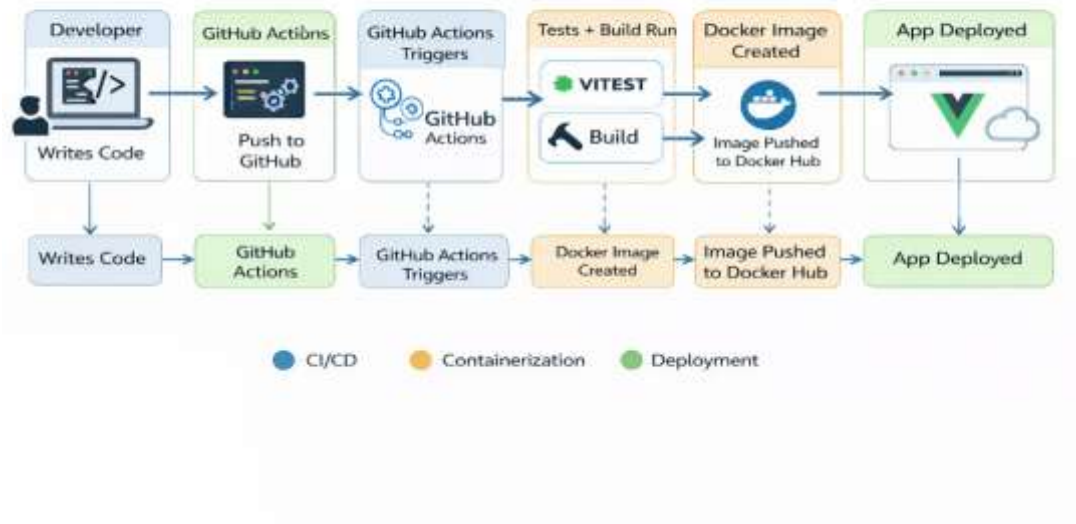
2.3 Information flow

The information flow represents how data moves within the system:

- User inputs are received as HTTP requests
- Requests are routed through Kubernetes Service to application Pods
- The application processes the input and generates output
- Output is returned as HTTP responses

All communication happens over standard HTTP protocol. Kubernetes ensures that requests are routed to active containers, maintaining reliability and availability.

2.4 Components Design



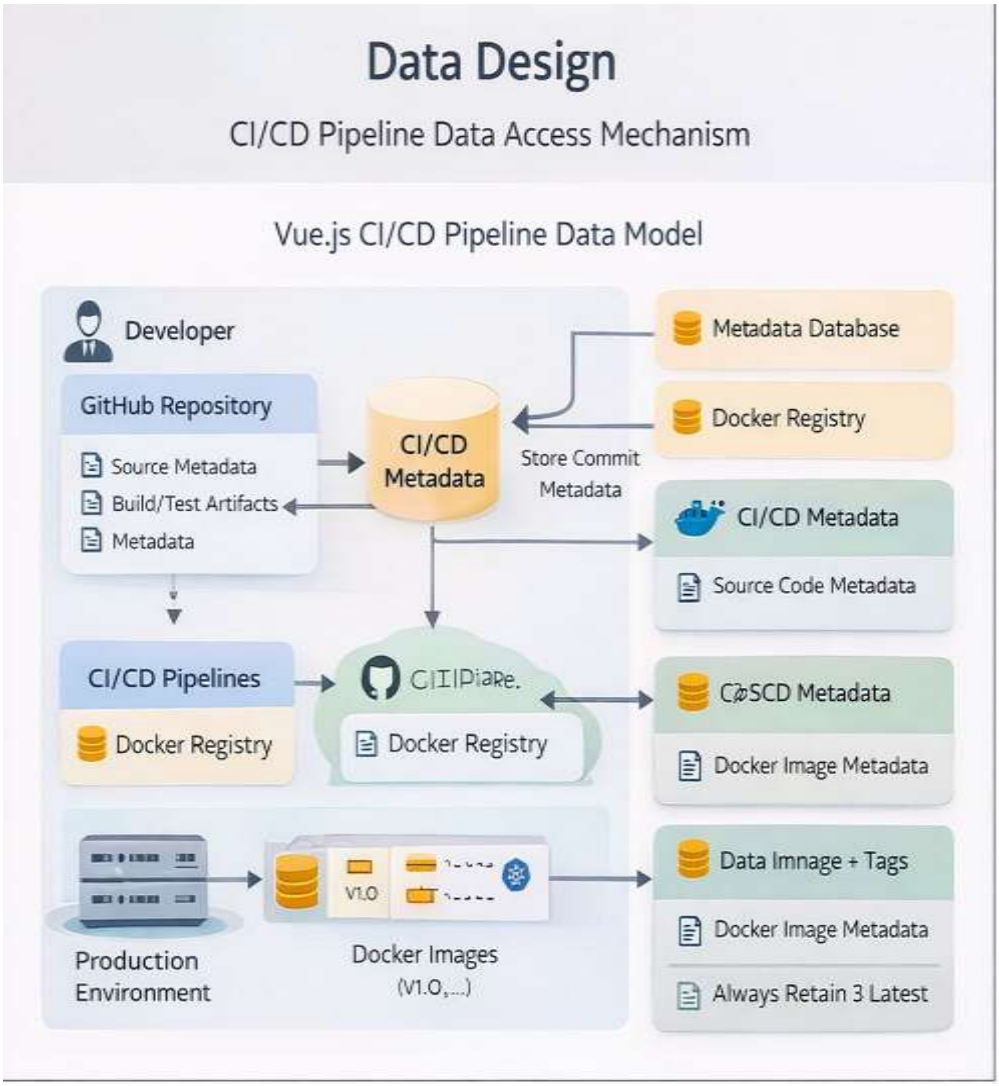
2.5 Key Design Considerations

The system is designed with the following considerations:

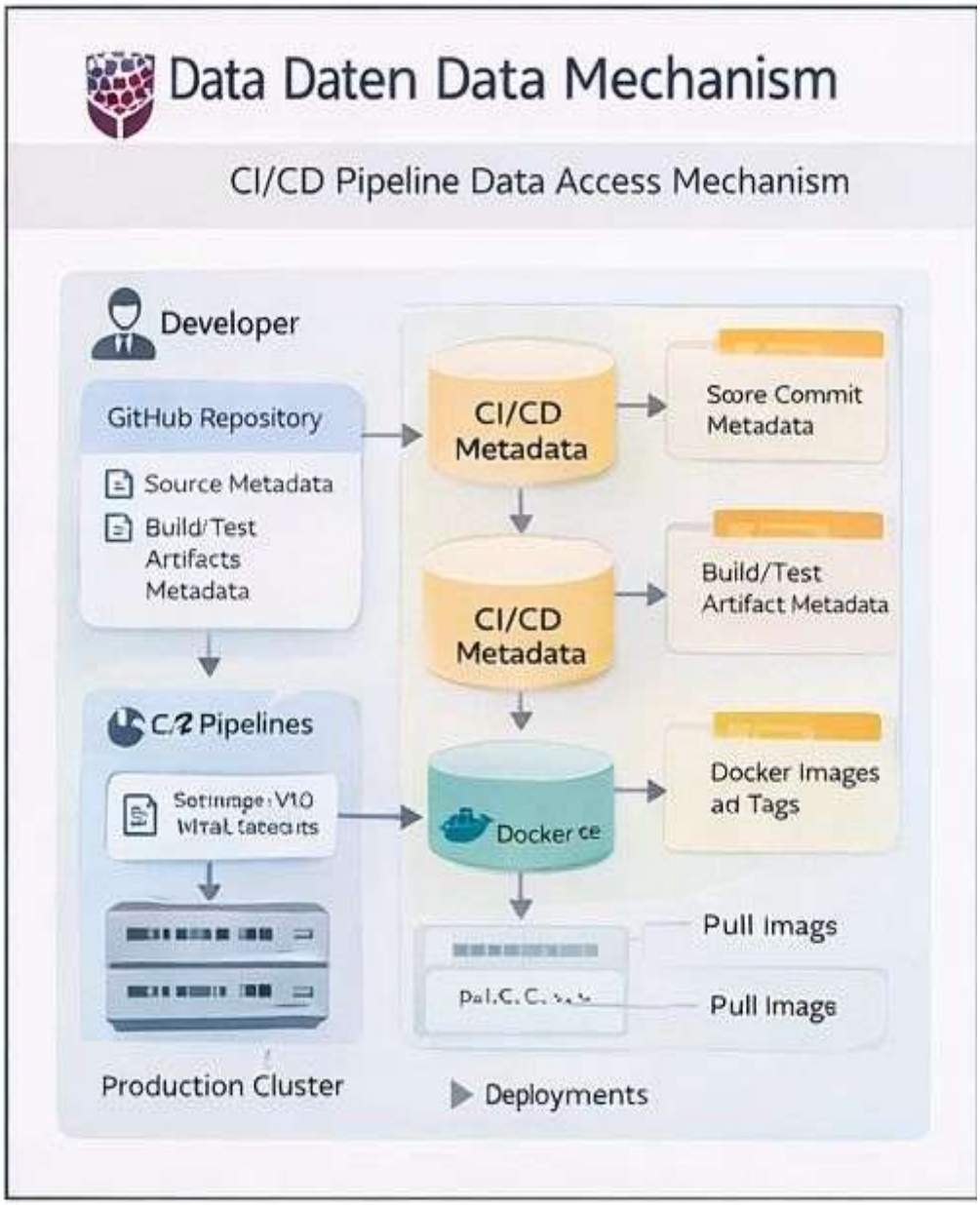
- **Scalability:** Kubernetes allows multiple replicas of the application to run simultaneously
- **Reliability:** Automatic restart of failed containers ensures high availability
- **Portability:** Docker ensures the application can run in any environment
- **Simplicity:** Lightweight Flask framework keeps the application easy to manage
- **Accessibility:** NodePort service allows external users to access the application
- **Separation of Concerns:** Each component handles a specific responsibilit

3.Data Design

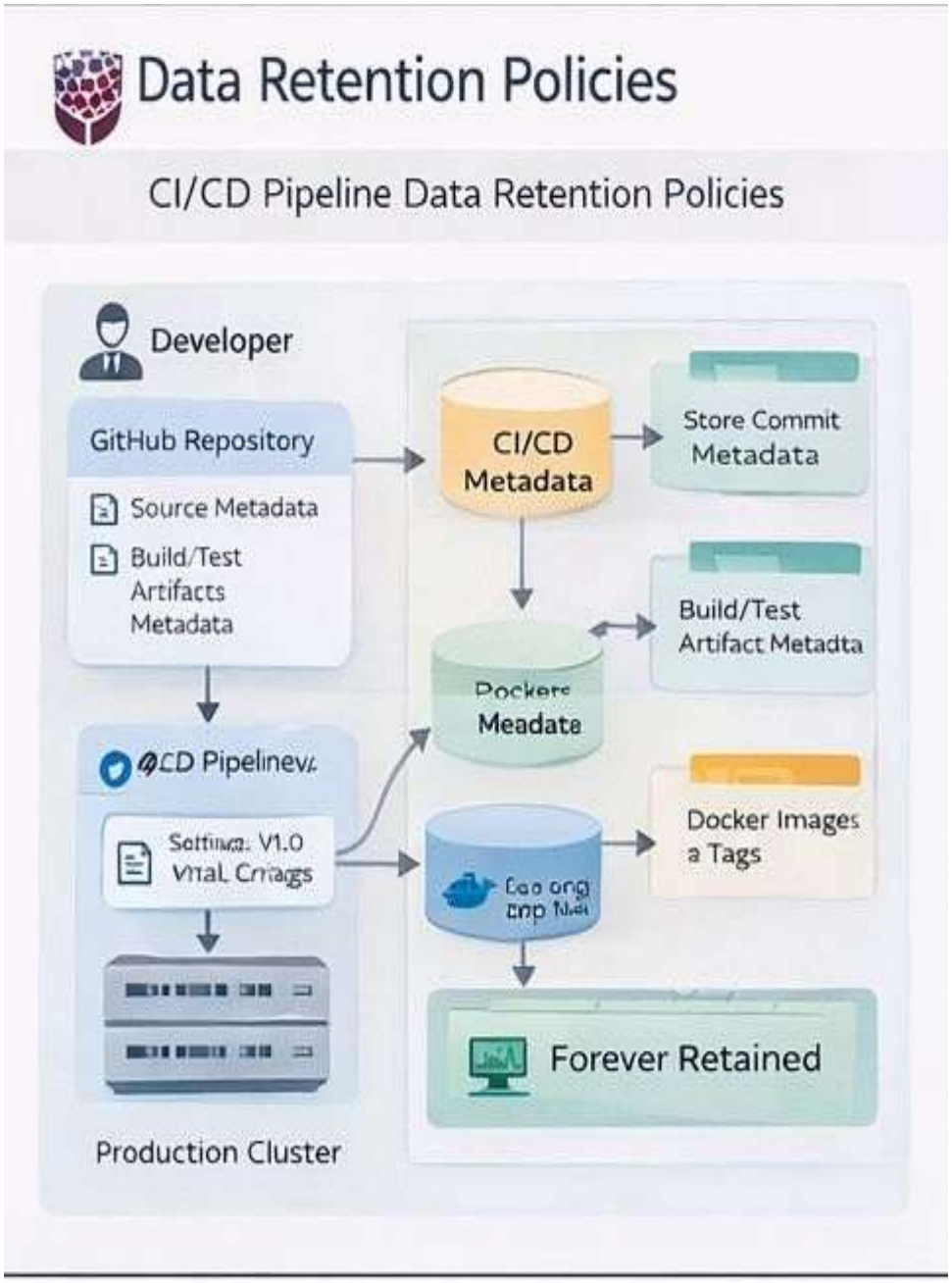
3.1 Data Model



3.2Data Access Mechanism



3.3Data Retention Policies



3.4 Data Migration

Data migration refers to the process of transferring application data from one environment to another, such as from a local system to a cloud-based deployment.

In the current system, the application is lightweight and does not rely on a dedicated database; therefore, large-scale data migration is not required. The primary migration involves transferring the application code and configuration files from the local development environment to the cloud environment.

This process includes:

- Uploading project files from the local machine to the EC2 instance
- Building a container image using Docker
- Deploying the container using Kubernetes

Since the application follows a stateless design, no persistent data needs to be migrated between containers. This simplifies the deployment process and reduces complexity.

For future enhancements, if the system integrates a database, data migration strategies may include:

- Database backup and restore
- Use of cloud-based storage services
- Automated migration scripts
- Data synchronization tools

4. Interfaces

Interfaces define how different components of the system interact with each other and how users interact with the application.

1 User Interface

The user interface is web-based and accessed through a browser.

- Users interact with the application via web pages rendered using Flask templates
- Input is provided through forms or URL requests
- Output is displayed as HTML pages

The interface is simple, responsive, and designed for ease of use.

2 Application Interface (API Interface)

The application exposes HTTP endpoints for handling requests.

- Communication follows standard HTTP/HTTPS protocols
- APIs are defined using Flask routes
- Requests include GET and POST methods

Example interfaces:

- / → loads homepage
- /submit → handles user input

3 Container Interface

The application runs inside a Docker container, which exposes a specific port (5000).

- Docker maps container port to host port
- Ensures the application is accessible outside the container
- Provides environment consistency across deployments

4 Kubernetes Interface

Kubernetes manages communication between components using:

- **Deployment:** Defines how the application is deployed and scaled
- **Service (NodePort):** Exposes the application to external users
- **Pods:** Run the containerized application

The Kubernetes Service acts as an interface between external users and internal application Pods.

5 External Interface

The system interacts externally with:

- Web browsers for user access
- Cloud infrastructure provided by Amazon Web Services
- Network interfaces defined through EC2 security groups

5. State and Session Management

State and session management define how user-specific data and application data are handled across multiple requests in the system. Since HTTP is a stateless protocol, additional mechanisms are required to maintain continuity between user interactions.

In this system, the application is built using the Flask framework and deployed in a containerized environment using Docker and Kubernetes. The management of state is designed to be simple and suitable for a scalable architecture.

5.1 Application State

The application is designed to be largely stateless, meaning that each request from the client is independent and does not rely on stored server-side data.

Key characteristics:

- No persistent server-side storage is required for handling requests
- Each request is processed independently
- Application logic does not depend on previous interactions

This stateless design improves scalability, as multiple instances (Pods) can handle requests without synchronization issues.

5.2 Session Management

For basic user interaction, Flask provides built-in session management mechanisms. Sessions are used to temporarily store user-specific data such as login status or input data.

- Sessions are stored using secure cookies on the client side
- Each session is identified by a unique session ID
- Data is encoded and sent with each request

This allows the application to maintain continuity without storing large amounts of data on the server.

5.3 Session Handling in Containerized Environment

Since the application runs inside containers managed by Kubernetes:

- Multiple Pods may handle requests for the same user
- Sessions must be lightweight and independent of a single container
- Client-side session storage ensures that any Pod can process the request

This avoids issues where session data is lost if a container stops or is restarted.

5.4 Stateless Deployment Advantage

The use of stateless design provides several benefits:

- **Scalability:** New Pods can be added without affecting existing sessions
- **Reliability:** Failure of one container does not impact overall user experience
- **Load Distribution:** Requests can be routed to any available Pod

This aligns with Kubernetes architecture, where containers are dynamically managed.

5.5 Future Enhancements (Optional State Management)

For more advanced use cases, the following improvements can be implemented:

- Use of centralized session storage (e.g., Redis)
- Database-backed session management
- Token-based authentication (JWT)
- Persistent user data storage

These enhancements can provide better control over session persistence and security in large-scale applications.

6. Caching

Caching is used to improve application performance by temporarily storing frequently accessed data so that repeated requests can be served faster.

In the current system, caching is minimal because the application is lightweight and primarily designed for demonstration purposes. However, basic caching mechanisms are inherently supported at different levels of the system.

At the application level, Flask can cache frequently rendered responses to reduce processing time. At the infrastructure level, container reuse and Kubernetes Pod lifecycle management help reduce startup overhead for repeated requests.

Although dedicated caching layers such as Redis are not implemented in this system, the architecture supports future integration of external caching solutions. This can significantly enhance performance for applications handling large volumes of traffic or repeated data queries.

Key benefits of caching include:

- Reduced response time

- Lower server load
- Improved user experience
- Efficient utilization of system resources

7.Non-Functional Requirements

Non-functional requirements define the quality attributes of the system, such as performance, security, reliability, and scalability. These requirements ensure that the system operates efficiently under various conditions.

7.1 Security Aspects

The system incorporates basic security practices suitable for a cloud-based deployment environment.

- **Network Security:** Access to the application is controlled using EC2 security groups, allowing only specific ports (e.g., 5000 or NodePort) to be accessed externally
- **Isolation:** Each application runs inside a container using Docker, providing process-level isolation
- **Orchestration Security:** Kubernetes ensures controlled deployment and management of containers
- **Data Protection:** Session data is stored securely using client-side cookies
- **Controlled Access:** Only authorized users with access to the EC2 instance can manage deployments

For production environments, additional security measures such as HTTPS, authentication, role-based access control (RBAC), and secret management can be implemented.

7.2 Performance Aspects

The system is designed to provide efficient performance under normal usage conditions.

- **Scalability:** Kubernetes allows horizontal scaling by increasing the number of Pods (replicas)
- **Load Distribution:** Incoming requests are distributed across multiple Pods using Kubernetes Service
- **Fast Deployment:** Docker containers ensure quick startup and consistent runtime behavior
- **Resource Efficiency:** Lightweight Flask framework minimizes resource consumption
- **High Availability:** Kubernetes automatically restarts failed containers to maintain uptime

These features ensure that the application remains responsive and reliable even with increasing user requests.

8. References

The following technologies and platforms were used as references for designing and implementing the system:

- Docker – for containerization of the application
- Kubernetes – for orchestration and deployment management
- Amazon Web Services – for cloud hosting using EC2
- Flask – for developing the web application
- Official documentation of Docker, Kubernetes, and Flask
- Online tutorials and technical resources related to containerization and cloud deployment